

Cells on Autopilot: Adaptive Cell (Re)Selection via Reinforcement Learning

Marvin Illian*, Ramin Khalili[†], Antonio A. de A. Rocha[‡], Lin Wang*

*Paderborn University, Germany

[†]Huawei Technologies, Germany

[‡]Fluminense Federal University, Brazil

Abstract—The widespread deployment of 5G networks, together with the coexistence of 4G/LTE networks, provides mobile devices a diverse set of candidate cells to connect to. However, associating mobile devices to cells to maximize overall network performance, a.k.a. cell (re)selection, remains a key challenge for mobile operators. Today, cell (re)selection parameters are typically configured manually based on operator experience and rarely adapted to dynamic network conditions. In this work, we ask: Can an agent automatically learn and adapt cell (re)selection parameters to consistently improve network performance? We present a reinforcement learning (RL)-based framework called CellPilot that adaptively tunes cell (re)selection parameters by learning spatiotemporal patterns of mobile network dynamics. Our study with real-world data demonstrates that even a lightweight RL agent can outperform conventional heuristic reconfigurations by up to 147%, while generalizing effectively across different network scenarios.

I. INTRODUCTION

The widespread adoption of 5G networks, coupled with the continued operation of 4G/LTE infrastructure, has significantly expanded the range of connectivity options for mobile devices [1], [2]. This coexistence creates a heterogeneous environment where multiple cells¹—potentially operating on different frequency bands—are simultaneously accessible to User Equipments (UEs) [3]. Such an environment provides unprecedented flexibility in cell (re)selection in idle mode², enabling mobile devices to dynamically choose between cells based on signal strength and other mobility-related factors [5]. While mobile operators gain the opportunity to optimize connectivity decisions to improve network efficiency and user experience, this also introduces significantly higher complexity in designing effective cell (re)selection strategies.

Despite the offered flexibility, most mobile operators still rely on fixed parameter configurations for cell (re)selection. These parameters, including various thresholds and priorities, are typically determined through offline planning and are seldom adapted in real time to reflect changing network conditions [5], [6]. While such static configurations simplify network management, they often fail to exploit the full potential of heterogeneous mobile network deployments, leading to suboptimal network utilization and uneven load distribution.

¹We define a cell as a sector of a base station that operates on a specific frequency band, covering a specific area of the network.

²Idle-mode cell selection plays a critical role in multi-carrier network deployments, enabling efficient load balancing among users without complex signaling. It reduces overhead and errors associated with connected-mode load balancing, ensuring smoother user transitions to active mode [4].

Adaptive parameter configuration for cell (re)selection poses significant challenges due to multiple reasons. First, to be compatible with existing network infrastructure and operations with minimal changes, cell (re)selection decisions cannot be made explicitly and should be the outcome of parameter configurations implicitly. The parameters form a huge and high-dimensional space, integrating several thresholds and priorities that exhibit non-linear interactions. Moreover, the parameters concentrate mostly on signal strength and are not directly performance-oriented. Hence, it is difficult to associate a particular parameter configuration to a performance level, making parameter reconfiguration intrinsically a hard problem.

Second, mobile networks are highly dynamic in nature, both spatially and temporally. Variations in user mobility patterns, traffic demand fluctuations, and interference levels make it difficult to determine optimal parameters in real time. In a highly heterogeneous network, the impact of changing one parameter can cascade through the network, affecting network utilization and load distribution. Traditional profiling- and experience-based optimizations struggle to cope with this scale and complexity and typically require extensive manual efforts [5]. We conjecture that a learning-based approach is more suitable. A learned agent can autonomously explore the high-dimensional parameter space, learning policies that dynamically adapt to changing network conditions and improving network performance and efficiency.

In light of the above challenge, we propose CellPilot, a neural adaptive cell (re)selection framework for mobile networks. CellPilot adopts reinforcement learning (RL) to intelligently optimize the parameters for cell (re)selection under dynamic user connectivity and mobility. The key innovation of CellPilot lies in its state and reward design, where UE status and network load conditions, both historical and current ones, are encoded to capture the dynamic network conditions. The reward function is carefully crafted to incorporate multiple objectives. CellPilot introduces a series of optimization techniques to enhance RL agent generalizability, ensuring robust performance under diverse traffic patterns and across network scenarios without retraining. The combination of principled RL modeling and tailored optimizations enables CellPilot to outperform the conventional heuristic-based approach and adapt seamlessly to evolving network environments.

Overall, we make the following contributions: After introducing the cell (re)selection problem (§ II), we

- propose CellPilot, a learning-based framework for adap-

tive tuning for cell (re)selection, leveraging reinforcement learning to capture the dynamic network environment spatiotemporally (§ III-A).

- introduce, adapt, and integrate several optimization techniques to improve the performance and generalizability of CellPilot (§ III-B).
- build a simulator with real-world network data and comprehensively evaluate CellPilot under varying conditions (§ IV). Our results show that CellPilot achieves up to 147% performance improvements. CellPilot also shows impressive generalizability, despite being lightweight.

We discuss related work in § V and conclude in § VI.

II. THE CELL (RE)SELECTION PROBLEM

In this section, we provide the background information and formally describe the cell (re)selection problem.

A. Cell (Re)Selection Process

The cell selection process in a mobile network [4], [7], [8] is detailed as follows. The process is identical for 4G/LTE and 5G, with only minor parameter differences. Each cell is powered by a base station operating on a specific carrier frequency and is identified by a unique cell ID [9]. When a UE powers up and enters a network, it performs cell selection and the selection criteria are based on Reference Signal Received Power (RSRP) and broadcast parameters.

Before a UE considers any cell for camping, it verifies if the signal fulfills the suitability condition. This requirement ensures that the UE can maintain adequate communication with the selected cell and is expressed as

$$S_{\text{rxlev},n}^{C_i} = \gamma_n^{C_i} - Q_{\text{rxlevmin}}^{C_i} > 0, \quad (1)$$

where $\gamma_n^{C_i}$ represents Received Signal Strength Indicator (RSSI) for UE n on cell C_i , $Q_{\text{rxlevmin}}^{C_i}$ denotes the minimum required RSSI for cell C_i as broadcast in the System Information Block (SIB), and $S_{\text{rxlev},n}^{C_i}$ defines the cell (re)selection received level for UE n on cell C_i .

Once the suitability condition is satisfied, the UE scans for all available cells, selects the most appropriate one based on network-defined criteria and camps on it. The UE then monitors the SIBs to stay informed about network configuration changes and neighboring cells. The network then guides idle-mode (re)selection through two complementary strategies: hierarchical cell (re)selection and equal priority cell (re)selection, as detailed in the following.

B. Hierarchical Cell (Re)Selection Strategy

In a hierarchical cell (re)selection strategy, the network assigns different priorities to cells. This allows the network to steer UEs towards a cell that provides best possible service by avoiding highly-congested cells even if they have a stronger signal. When the serving cell C_i has a lower (re)selection priority than a neighboring cell C_j , the reselection criteria focus on the RSRP of the neighboring cell. The reselection process is initiated when the RSRP of C_j exceeds a threshold

$T_{\text{XHigh}}^{C_i \rightarrow C_j}$ for the duration of the reselection timer $T_{\text{reselection}}$ to avoid frequent reselections if brief fluctuations occur:

$$\gamma_n^{C_j} > T_{\text{XHigh}}^{C_i \rightarrow C_j} \text{ for duration } T_{\text{resel}}. \quad (2)$$

If multiple cells meet the criterion, the UE will reselect the highest-ranked cell among the highest priority frequencies.

Reselection to a neighboring cell with a lower priority involves multiple criteria, and works against the network's preferred hierarchy. For inter-frequency reselection, the process is initiated when the serving cell's RSRP $S_{\text{rxlev}}^{C_i}$ falls below a threshold S^{Inter} . This process is performed analogously for intra-frequency reselection using S^{Intra} . Subsequently, reselection to a lower priority cell is triggered if both the serving cell's RSRP $\gamma_n^{C_i}$ and the neighboring cell's RSRP $\gamma_n^{C_j}$ meet the following conditions for the duration of the reselection timer T_{resel} :

$$\begin{aligned} \gamma_n^{C_i} &< T_{\text{SLOW}}^{C_i} \text{ for duration } T_{\text{resel}}, \\ \gamma_n^{C_j} &> T_{\text{XLow}}^{C_i \rightarrow C_j} \text{ for duration } T_{\text{resel}}. \end{aligned} \quad (3)$$

C. Equal Priority Cell (Re)Selection Strategy

Equal priority cell (re)selection is employed when the RSRP difference between cells is less than 3 dB. In this case, the absolute RSRP levels become not as critical and the focus shifts to relative differences. This strategy directs users to cells with stronger RSRP while allowing network operators to fine-tune the traffic distribution. The offset parameter $Q_{\text{Offsetfreq}}^{C_i \rightarrow C_j}$ can be dynamically adjusted by the network to influence the UE reselection process. By increasing this offset, the network can discourage UEs from selection to certain neighboring cells, effectively balancing traffic according to prevailing network conditions and meeting congestion or quality of service targets.

The reselection in equal priority applies ranks to the serving and neighboring cells based on their RSRP and the offset parameter. The rank of the neighboring cell is defined as

$$R_{\text{Nbr}} = \gamma_n^{C_j} - Q_{\text{Offsetfreq}}^{C_i \rightarrow C_j}. \quad (4)$$

To avoid frequent reselections, the rank of the serving cell is adjusted with a hysteresis value $Q_{\text{Hyst}}^{C_i}$ and is defined as

$$R_{\text{Ser}} = \gamma_n^{C_i} + Q_{\text{Hyst}}^{C_i}. \quad (5)$$

Again, a timer $T_{\text{reselection}}$ ensures that the reselection decision is robust to brief fluctuations in RSRP.

$$R_{\text{Nbr}} > R_{\text{Ser}} \text{ for duration } T_{\text{resel}}. \quad (6)$$

D. Conventional Heuristic Baseline

As a reference point for performance comparison, we use the network operator's conventional heuristic parameter set deployed in Petrópolis, which is also the evaluation region in our study. This configuration is specifically tuned for the Petrópolis area, ensuring that our RL approach is not compared against a weak or untuned baseline. (See Table I).

III. CELLPILOT DESIGN

In this section, we model the learning problem and introduce CellPilot, our proposed learning framework based on RL.

TABLE I
CONVENTIONAL HEURISTIC REFERENCE VALUES

Symb.	$Q_{\text{Hyst}}^{C_i}$	$Q_{\text{Offset}_{\text{freq}}}^{C_i \rightarrow C_j}$	$Q_{\text{rxlevmin}}^{C_i}$	$T_{\text{XHigh}}^{C_i \rightarrow C_j}$	$T_{\text{XLow}}^{C_i \rightarrow C_j}$	$T_{\text{SLow}}^{C_i}$
Val.	3 dB	14 dB	-60 dBm	-56 dBm	-58 dBm	-54 dBm

TABLE II
CELL (RE)SELECTION PARAMETERS, RANGES, AND PURPOSES

Symbol	Range (dB/dBm)	Purpose
$T_{\text{XHigh}}^{C_i \rightarrow C_j}$	$[-100, 0]$	Thresh. to reselect to higher priority cell
$T_{\text{XLow}}^{C_i \rightarrow C_j}$	$[-100, 0]$	Thresh. to reselect to lower priority cell
$T_{\text{SLow}}^{C_i}$	$[-100, 0]$	Serving cell signal strength reselection threshold
$Q_{\text{Hyst}}^{C_i}$	$[0, 30]$	Hysteresis to stabilize serving cell selection
$Q_{\text{Offset}_{\text{freq}}}^{C_i \rightarrow C_j}$	$[0, 30]$	Offset biasing reselection btw. equal priority cells
$Q_{\text{rxlevmin}}^{C_i}$	$[-100, 0]$	Min. required signal level for cell suitability

A. Problem Modeling

We assume a discrete-time system where time is divided into small steps. A UE can change its state at every step. For the ease of exposition we first assume that an RL agent makes an observation of the environment and takes an action at every step. Later, we will show how the agent can be generalized to other cases with different temporal dynamics.

Observation space. The RL agent operates under partial observability, as it omits variables like per-UE mobility, IDLE/ACTIVE timers, and reselection counters. Formally, the control task is modeled as a Partially Observable Markov Decision Process (POMDP). The observation space for the agent consists of per-cell features and global metrics. Per-cell features include the normalized available bandwidth and the number of connected UEs in ACTIVE mode normalized by the total number of UEs, including the ones camping on it in IDLE mode. Global metrics include the mean and standard deviation of available bandwidth across all cells, the mean and standard deviation of the normalized number of connected UEs per-cell, and the ratio of UEs in IDLE mode to the total number of UEs. These global metrics compress the per-cell data into a fixed-size representation, providing a bird's-eye view of the network state. This fixed-size representation allows the policy to generalize across varying network topologies, as it decouples global network state representation from the size.

Action space. For each of the cell (re)selection parameters (see Table II), the model outputs the mean μ_i and standard deviation σ_i that are used to define a Gaussian distribution. The action is sampled from these Gaussian distributions, allowing the agent to adaptively adjust the (re)selection parameters. The action space is a continuous vector of size $2 \cdot 6$. The constant six comes from the six (re)selection parameters that are shared across all cells. The first half of the action vector contains the means, and the second half contains the standard deviations for all the (re)selection parameters. As the action controls both the mean and standard deviation, it allows the agent to adaptively adjust the exploration intensity by modifying the standard deviation. The resulting actions (in range $[0, 1]$) are then linearly mapped to the physical ranges for the parameters

(see Table II). We intentionally exclude cell priorities from the action space because they introduce discrete, high-impact changes in the reselection logic and are typically adjusted on much longer time scales. Cell priorities are fixed to the values used by the conventional heuristic baseline.

CellPilot assumes homogeneous cell parameters across a network region, reflecting current practice due to the complexity of manual configuration. Extending CellPilot to heterogeneous parameters is feasible by expanding state/action spaces or adopting a multi-agent approach, but is left for future work.

Neural network architecture. By keeping a flat, normalized input vector, we can employ a small Multi-Layer Perceptron (MLP) instead of more complex architectures, reducing the number of parameters. To keep the agent fast, we adopt a simple four-layer feedforward network. The input layer is designed to match the environment's observation space. Then, the network features a MLP with two hidden layers and $\tanh$ activation functions. The output layer uses a sigmoid activation function to ensure that the actions remain within a valid range for the action space. One inference pass through the network requires fewer than 5 MFLOPs, enabling sub-millisecond execution on commodity hardware.

Reward. The reward function is designed to encourage the agent to improve on the following three objectives: The *throughput* term rewards the agent for achieving higher total throughput of all UEs in the network. The *load balancing* term encourages the agent to equalize throughput across individual cells. The *per-UE efficiency* term is designed to maximize the throughput of each UE while considering the number of active UEs in the network. The agent receives higher reward, if per-UE throughput is high while more UEs are active, compared to where only few UEs are active, to avoid the agent focusing on a few UEs at the expense of others.

During training, the agent receives a reward at each step t based on its action and the environment's response. The reward is a composite function designed to balance throughput, load balancing, and per-UE efficiency. Specifically,

$$r_t = w_1 \cdot r_t^{\text{tput}} + w_2 \cdot r_t^{\text{bal}} + w_3 \cdot r_t^{\text{ue-eff}} \quad (7)$$

where $w_1 + w_2 + w_3 = 1$ are weighting coefficients. The throughput term is defined by

$$r_t^{\text{tput}} = \left(\frac{T_t}{B^{\text{tput}}(t)} \right) - 1 \quad (8)$$

where T_t is the total throughput, and $B^{\text{tput}}(t)$ is a moving average baseline estimator. The load-balancing term can be characterized by

$$r_t^{\text{bal}} = \left(\frac{B^\sigma(t)}{\sigma_t} \right) - 1 \quad (9)$$

where σ_t is the standard deviation of per-cell throughputs, and $B^\sigma(t)$ is the corresponding baseline estimator. This is in line with the goal to provide the same expected throughput in a given area that operators aim to achieve. Finally, the per-UE efficiency term is given by

$$r_t^{\text{ue-eff}} = \left(\frac{\overline{T}_{\text{UE},t}}{B^{\text{UE}}(t)} - 1 \right) \cdot \min \left(1, \frac{\overline{\text{UE}}_{\text{active},t}}{\text{UE}_{\text{max}}} \right) \quad (10)$$

where $\bar{T}_{\text{UE},t}$ is the mean per-UE throughput, $B^{\text{UE}}(t)$ is its baseline estimator, and $\bar{\text{UE}}_{\text{active},t}$ is the average number of active UEs during interval t and UE_{max} is the maximum number of UEs in the network. This formulation rewards the agent for load scenarios where many UEs are active, representing high-load network conditions.

Baseline estimators. To determine whether the agent is performing better or worse, we employ baseline estimators, computed separately for each of the reward terms, as detailed above in the reward function definition. The baseline estimators are used to compute the relative performance difference between the current agent performance and the baseline performance from the past. The calculation of the baseline estimators is explained in the following section. At initialization, when the baseline has not yet been populated, we use the performance of a conventional heuristic approach as a point of reference.

B. CellPilot Learning Algorithm and Optimizations

Due to space constraints, we summarize only the core generalizability mechanisms here; further details on cross-scenario, temporal, and episode-level generalization are provided in the full version [10]. The agent is trained on episodes, each defined as a set of consecutive steps in a specific environment. To ensure reproducibility, we seed the environment with a fixed random seed for each episode. The seeds are used for random number generator initialization and determine the stochastic processes which control the UEs' modes (IDLE/ACTIVE) and positions, effectively generating different network topologies and traffic patterns during training. The environment seed is used to generate a unique scenario, including the cell topology and UEs positions. Each UE receives a unique random number generator seed derived from the episode seed, based on which UE seeds the location and mode throughout the episode. UEs are placed in realistic positions on streets or inside buildings and cell tower positions correspond to real cell tower positions. For the mode switch, each UE generates state-transition events drawn from exponential distributions seeded with the UE-specific seed. During the training, we apply seed shuffling to prevent the agent from memorizing the order of the scenarios. For evaluation, we use seeds that are unknown to the agent during training, presenting unique scenarios to the agent. After each episode, we reset the environment.

During the development of the learning algorithm, we first experimented with the popular actor-critic method A2C [11], but it did not converge, likely due to the intrinsic dynamics of the problem. We hypothesize that instability arises from the fact that different seeds yield different total throughput. Each seed produces different temporal reward profiles, leading to non-stationary targets for the value function. Hence, the critic network tries to regress to a mixture of unrelated target values. Without a normalizing baseline, the advantage estimates vary significantly, leading to gradient interference. A similar problem is described in [12], where the limitations of value-based methods in scheduling contexts are mentioned. Specifically, it is highlighted that if the target of the value function becomes non-stationary, the learning can be hindered.

To avoid this problem, we adopt the vanilla REINFORCE algorithm [13] with seed-specific baseline estimators [14]. The REINFORCE algorithm does not use a shared critic network but computes the gradients based on the rewards received at each time step. Since rewards are normalized by the baseline estimators, gradient interference between seeds is avoided.

The baseline estimators compute the expected performance of the agent. For each seed s , a baseline $B_s(t)$ is computed as a moving average over the last w episodes at time step t . The episodes with the same seed share the same network topology and traffic patterns, making the baseline an unbiased estimator of the expected return for the current policy.

To stabilize training, we apply global gradient clipping using the ℓ_2 norm, preventing destabilizing updates [15]. To favor known regions of the action space, we initialize the network weights so that the initial mean actions reproduce the conventional heuristic parameter set. We accomplish this by writing the values into the output-layer biases of the neural network. This warm start helps the agent to quickly adapt to the environment and improve initial performance [16]. During training, naturally, the agent's performance fluctuates. To retain the best-performing model, we implement a checkpointing mechanism that periodically saves the model. We determine convergence during training from the episode rewards. If the reward approaches zero, i.e., the agent cannot improve on top of history performance, we conclude training convergence.

Curriculum learning. Initially, the agent may take poor actions, leading to sub-optimal resource allocation, not only for UEs that switch to ACTIVE mode right after, but also for UEs that become active later as the resources are blocked until released by the UEs. In such cases, the agent may not be able to correct its previous errors. Even worse, the agent has to take sub-optimal actions based on the sub-optimal state, as such earlier errors may lead the policy into regions of the state space from which recovery is difficult or impossible. To mitigate this, we employ a powerful technique called curriculum learning [17], starting with shorter episodes which provide a simpler environment. The agent learns to take good actions from such shorter episodes, stabilizing early learning and avoiding sub-optimal states. We then gradually increase the complexity of the environment by extending the episode length, allowing the agent to learn to take good actions from longer episodes. During the training, the learning rate is set to 10^{-4} at the beginning of training and is adjusted by halving it with every new curriculum learning round.

History stacking. As the problem is modeled as a POMDP, we augment the observation space with a history of the last k full observations to improve the state inference. This history stacking allows the agent to disambiguate otherwise identical state observations, such as the same cell-load vector arising from rising versus falling load trends. This reduces partial observability, improving decision making as long-term trends can be inferred from the history [18]–[20].

IV. EVALUATION

We evaluate the performance of CellPilot with real-world mobile network scenarios and aim to answer: (1) What are the performance gains of CellPilot compared to the conventional heuristic approach? (2) How well does CellPilot generalize to new, unseen network scenarios? We report the main performance and generalization results here; a more extensive ablation analysis is available in the full version [10].

A. Evaluation Setup

We build a Python simulator implementing the cellular network model described in Section II. The simulator uses line-of-sight geometry and a sectorized antenna pattern with a 120° -sector width—inferred from the spatial layout and visual coverage of cells, while azimuths and positions were provided by the operator—and a 10 dB gain for the main lobe [21], with a 0 dB signal elsewhere as a simplifying modeling choice. The free-space path-loss model is used at GHz frequencies, and we optionally include a light building obstruction loss of 6 dB [22] for each wall intersecting the line-of-sight between a cell and a UE. Thermal noise is modeled as $-174 \text{ dB mW Hz}^{-1}$ (Johnson-Nyquist noise [23]), and the spectral efficiency is computed using a nearest-neighbor lookup based on the SINR-to-SE mapping [24], assuming a noise-limited environment. We use a resource-fair scheduling algorithm that allocates bandwidth equally to all active UEs in a cell, iteratively ensuring fairness using a water-filling algorithm.

Our experiments are conducted over multiple distinct geographical areas, each with a unique topology. The cell tower information is extracted from a real-world dataset obtained from a major mobile operator in Brazil. The dataset provides exact locations, and deployment configurations such as azimuth, beamwidth, frequency, bandwidth, and reselection parameters. Since not all reselection parameters are available in the dataset, we infer representative values for missing parameters sourced from publicly available material [25]–[27] that are not specified for the conventional approach and the warm start of CellPilot. Industry guideline [28] validates our approach of deriving default parameters from multiple authoritative sources, including vendor specifications and industry best practices, when complete operator configuration datasets are unavailable. We use the following three geographical areas located in Petrópolis, Brazil in our experiments.

- **Baseline area** consists of two cell towers, 15 cells, and is of size 2.36 km^2 .
- **Alternative area** is used for evaluation only and has the same number of cell towers, number of cells, and area size as the baseline area, but is geographically distinct.
- **Large area** is significantly larger with six cell towers (see Figure 1), 48 cells, and is stretched across 6.10 km^2 . This area is used to evaluate CellPilot scalability.

Temporal dynamics. We evaluate the RL agent under different temporal dynamics to assess its robustness and adaptability. The baseline model is trained and evaluated with a Parameter-Refresh Interval (PRI) of one, resulting in a

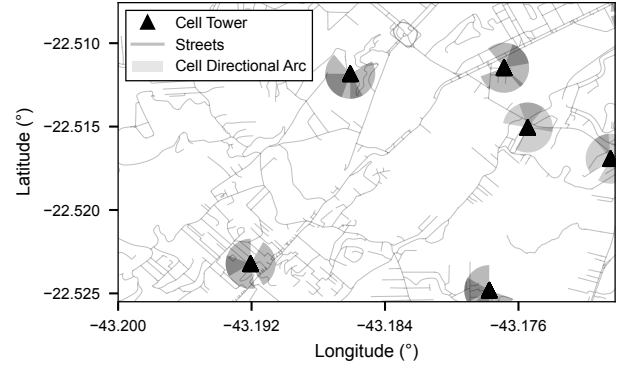


Fig. 1. Large network deployment area used for training and evaluation [29].

reference Update-to-Dwell Ratio (UDR) of approximately 0.2. The *Stress Test* scenario evaluates the baseline model with a PRI of 10 and a UDR of approximately 2. This scenario tests the robustness of the RL agent against previously unseen temporal dynamics, where on average two UE mode changes can occur between two updates. The *Synchronous Updates* scenario trains the model with a PRI of 5 to match the empirical UE State Dwell Time (SDT), resulting in a UDR of approximately one. In the *Slow Updates* scenario, the model is trained and evaluated with a PRI of 10 to consistently simulate a slow control loop, resulting in a UDR of approximately 2.

UE setup. The UEs are placed at valid street or indoor positions, ensuring realistic placement; the simulator also supports building obstruction loss, which is disabled during training for efficiency. The UEs' mode switch is modeled as a Poisson process with exponential interarrival times parameterized with rates $\lambda_{\text{IDLE}} = \lambda_{\text{ACTIVE}} = 0.2$ leading to an average SDT of 5 s. UE mobility is supported but disabled, as CellPilot does not consider handovers. However, the agent is trained on a variety of scenarios with unique UE positions. The training set consists of 100 unique scenarios, each with 500 UEs. The UE positions and traffic patterns are seeded randomly, ensuring that the agent is exposed to a diverse set of conditions. The validation is performed against 100 unseen scenarios, which allows us to evaluate the generalizability of the agent across different UE distributions and traffic patterns.

Model and system parameters. The models use 1,024 neurons per layer with 2 hidden layers; smaller models (e.g., 512) led to unstable training and suboptimal performance. To ensure stability, we apply gradient clipping with a threshold of 10. The action space ranges are normalized to the values in Table II. The weights of the reward components are assigned as follows: $w_1 = 0.4$ for throughput, $w_2 = 0.4$ for load balancing, and $w_3 = 0.2$ for per-UE efficiency. For the two scenarios—the scalability evaluation on the large area (Figure 5) and the synchronous updates case in the temporal dynamics evaluation (Figure 6)—we employ reward weights $w_1 = 0.025$, $w_2 = 0.95$, and $w_3 = 0.025$. In these scenarios, the agent achieved proportionally larger throughput gains; hence, we compensate with a higher load-balancing weight to encourage more balanced performance improvements. To limit exploration, we use a Gaussian distribution with a standard

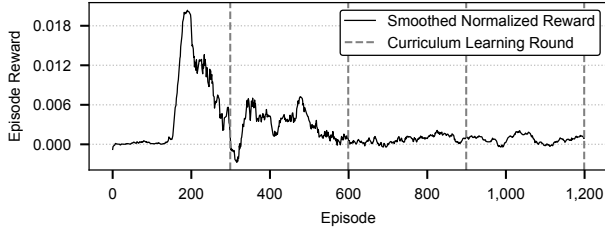


Fig. 2. Achieved total episode reward over episodes using the baseline area.

deviation capped at 0.1 for the action space. Curriculum learning starts at 30s, and is incremented by 10s after 3 passes through all environment seeds. The increase is applied 4 times in total leading to a final episode length of 50 s. History stacking is applied, where the last $k = 10$ observations are included in the observation vector.

B. Overall Performance & Generalizability

We first evaluate CellPilot performance (1) across the three geographical areas (2) under load scaling from 100 to 1,000 UEs, and (3) over four different temporal dynamics from low-latency to slow control loops (*Baseline*, *Stress Test*, *Synchronous Updates*, and *Slow Updates*), thus probing spatial, load-dependent and temporal generalization capabilities of our approach. For each evaluation we confront the agent with 100 unique scenarios unseen during training with 500 UEs with a fixed episode length of 60 seconds. To investigate load scaling, we evaluate the agent on the baseline area with 100 to 1,000 UEs in increments of 100. We use the conventional approach as a baseline for comparison using the reference values in Table I. For easier visual interpretation, the load-balancing component in the reward term—defined as a (normalized) standard deviation where lower is better—is computed as its reciprocal $1/r_t^{\text{bal}}$, so that positive values denote improvement.

Convergence. Due to the moving-average estimators, identical performance yields lower rewards over time, requiring continuous improvement to maintain reward levels. The agent’s reward naturally decreases over training and approaches zero at convergence. Although this can be ambiguous as phases of zero reward may still be followed by further learning. To decide on convergence, we check throughput and load balancing performance for models at different stages. We observed that sometimes the agent gets stuck at zero reward for a while but then learns further, optimizing only one metric. Throughput gains are relatively easier to achieve and larger. Therefore, we adapt reward weights to compensate in certain scenarios. Figure 2 shows the episode reward normalized by the episode length. Vertical lines indicate the curriculum learning rounds, where the episode length increases. Training stabilizes after the second round, indicating convergence.

UE scaling. We quantify how performance varies with network load by fixing the agent trained on 500 UEs and evaluating it on $N = 100, 200, \dots, 1000$ UEs in the baseline area. Figure 3 shows the components of the reward function separately as average values over the number of UEs in the system. Despite being trained on 500 UEs, the agent can generalize to different numbers of UEs, showing a gain

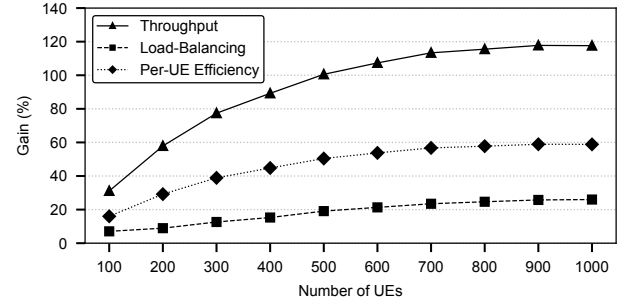


Fig. 3. Average values for reward components for the baseline area over different numbers of UEs in the system. The agent is trained on 500 UEs.

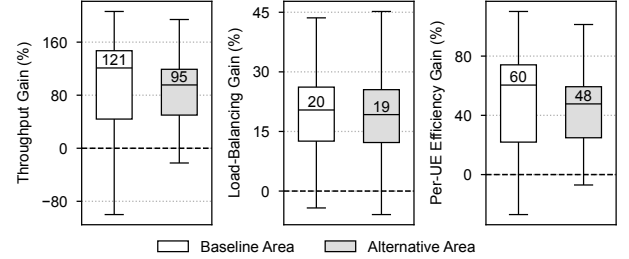


Fig. 4. Performance gains for the baseline area and alternative area. The agent is trained on the baseline area and evaluated on both areas to show the cross-area generalizability.

for all components of the reward function. It consistently shows better average throughput, load balancing, and per-UE efficiency compared to the conventional heuristic across all evaluated numbers of UEs. In general, the gains are lower for smaller N . Load balancing improves up to 26% for $N = 1000$ compared to the conventional heuristic. Average total throughput consistently improves with increasing number of UEs with a maximum gain of 118%. Average per-UE efficiency improves up to 59% for $N = 1000$ compared to the conventional heuristic. In general, higher numbers of UEs lead to diminishing returns for all three components. For throughput and per-UE efficiency, this is expected since the conventional heuristic increasingly saturates the available spectrum as UE count grows. Similarly, load balancing naturally improves with more UEs per cell, reducing the optimization potential.

Cross-area evaluation. We evaluate the baseline-trained agent on the baseline and alternative areas; the latter differs in topology and UE distribution but shares size, tower count, and cell count. Figure 4 shows that the agent outperforms the conventional heuristic in both areas across all metrics. The performance gains in the alternative area are comparable to those in the baseline area. The confidence intervals for throughput and per-UE efficiency are smaller in the alternative area, indicating more consistent performance than in the baseline area. This indicates that the agent has learned a policy that generalizes well to unseen areas with different cell topologies and UE distributions. We attribute this to the alternative area’s geography, where cells are more evenly distributed than in the baseline, leading to less diverse UE-to-cell associations. This shows the robustness of the policy to topology shifts.

Scalability to larger areas. To evaluate the scalability of policy, we train the agent on the large area with about $2.6 \times$

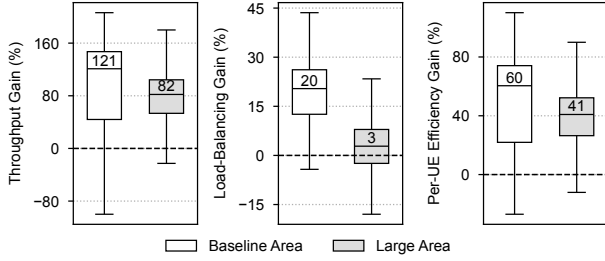


Fig. 5. Performance gains for the large area. The agent is trained on the large area and evaluated on it. The baseline-area model is shown for comparison to highlight the scalability of the approach.

the size of the baseline area and $3.2 \times$ the number of cells and $N = 500$ UEs. For this scenario, we increase the moving-average estimator window size from 2 to 10 to provide a more stable baseline for reward computation. We observed that with the smaller window size, the baseline prediction adapts too quickly to bad performance, resulting in rewards that remain close to zero and provide insufficient learning signal. In contrast, a larger window size maintains a higher baseline estimate that does not adapt as quickly, requiring the agent to achieve larger performance improvements to generate positive rewards and sustain learning progress. Figure 5 shows the results of the evaluation on the large area. The agent outperforms the conventional heuristic across all metrics with high confidence. The median value for total throughput gain is smaller at 82% compared to 121% in the baseline area. This is expected as with fewer UEs per cell the conventional heuristic already uses most available spectrum, hence, the agent has limited headroom to increase total throughput. The load-balancing component shows the modest improvement with a median value of 3% over the conventional heuristic, about 1/6 of the gain in the baseline area. This is expected as we use homogeneous (re)selection parameters across all cells, which limits the agent’s ability to fine-tune load balancing in a larger area with more cells. The per-UE efficiency gain is 41% (around two thirds of the baseline gain). Even in larger network topologies, our approach reliably improves over the conventional heuristic proving its scalability.

Diverse temporal dynamics. Figure 6 shows the results under different temporal dynamics. The *Stress Test* scenario shows a median of 56% in throughput gain, 7% in load balancing, and 27% in per-UE efficiency. The gains are lower than in the baseline scenario where the agent is evaluated under the same temporal dynamics as it was trained on. Intuitively this makes sense, as the agent learned to take action on shorter timescales and is now confronted with a slower control loop. The agent cannot react as quickly to traffic bursts and update network parameters. Compared to the conventional heuristic, however, the agent still manages to improve performance across all three metrics, indicating that it has learned a policy that is robust to changes in temporal dynamics. The *Slow Updates* scenario shows a median throughput gain of 139%, 20% load-balancing gain and 69% for per-UE efficiency yielding slightly higher gains in throughput and per-UE efficiency compared to the baseline scenario. These results

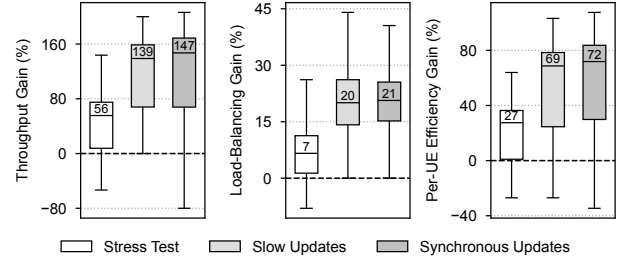


Fig. 6. Performance gains for different temporal dynamics. In the *Stress Test* scenario, the agent is trained with a $UDR > 1$ and evaluated with a $UDR < 1$. The *Slow Updates* scenario trains and evaluates the agent with a $UDR < 1$. The *Synchronous Updates* scenario trains the agent with a $UDR \approx 1$.

suggest that by training on a slower timescale, the agent learns to identify more resilient, long-term parameter configurations that generalize better across multiple user state transitions. The *Synchronous Updates* scenario shows a median gain of 147% in total throughput, 21% in load balancing, and 72% in per-UE efficiency. These results outperform all other temporal dynamics scenarios. This indicates that aligning the update interval with the timescale of UE transitions allows the agent to optimally time parameter updates, minimizing reactive lag and enhancing overall network performance.

V. RELATED WORK

Cell (re)selection in mobile networks. Cell (re)selection parameter optimization has received limited attention in the literature [4], [5], [30]. To mitigate the performance issue observed in [6], [30], researchers have explored device-assisted fixes [30]. Some works leverage large datasets to learn how to configure newly added radio channels or tune parameters for better performance [31]–[34]. However, these techniques typically rely on static configurations and, in some cases, require manual verification. The closest work to ours is [5], where they optimize the cell (re)selection parameters and propose a heuristic approach based on historical network loads. There are also studies focusing on handover optimizations [35]–[38] or band switching [3] for mobility management. To the best of our knowledge, we are the first to present a learning-based approach for adaptive parameter reconfiguration for cell (re)selection based on runtime network state.

RL for networking problems. Many networking problems are intrinsically complex involving dynamic environments, which renders RL a powerful tool for tackling them [39]. For example, researchers have applied RL-based solutions to problems such as video streaming [40], congestion control [41], and traffic scheduling [42]. For mobile network management, RL has been applied to problems like network slicing [43] and handover optimization [38]. So far, no work has explored RL-based parameter reconfiguration for cell (re)selection.

VI. CONCLUSION

We presented CellPilot that autonomously adapts idle-mode cell (re)selection parameters in heterogeneous 4G/5G networks. By modeling the problem as a partially observable Markov decision process and incorporating optimization techniques, CellPilot optimizes network performance under

dynamic conditions. Evaluation using a custom simulator with real-world topologies shows that CellPilot outperforms conventional heuristics by 147% in throughput with robust generalizability across unseen geographic areas, scales, loads, and control loop latencies without retraining. Compatible with existing 4G/5G specifications, CellPilot lays the groundwork for adaptive cell (re)selection, demonstrating the feasibility and benefits of transitioning from static configurations to RL-driven, autonomous network operations.

REFERENCES

- [1] Bundesnetzagentur. (2024) Bundesnetzagentur extends mobile communications map to include high-performance 5g standalone. [Online]. Available: https://www.bundesnetzagentur.de/SharedDocs/Pressemittelungen/EN/2024/20240613_MoFu.html
- [2] NTIA. (2022) NTIA establish spectrum coordination initiative. [Online]. Available: <https://www.ntia.gov/press-release/2022/fcc-ntia-establish-spectrum-coordination-initiative>
- [3] A. Hassan, W. Ye, A. Zhang, J. Carpenter, R. Zhu, S. Jin, F. Qian, Z. M. Mao, and Z. Zhang, "The case for boosting mobile application QoE via smart band switching in 5g/xg networks," in *ACM HotMobile*, 2024.
- [4] M. Malmirchegini, M. Shukair, P. Rached, S. Sawhney, M. Ambriss, K. R. Chaudhuri, and S. Sarkar, "Layer management through idle-mode parameter optimization in multi-carrier LTE networks," in *WCNC*, 2016.
- [5] Q. Li and C. Peng, "Reconfiguring cell selection in 4g/5g networks," in *IEEE ICNP*, 2021.
- [6] H. Deng, K. Ling, J. Guo, and C. Peng, "Unveiling the missed 4.5g performance in the wild," in *ACM HotMobile*, 2020.
- [7] "5G; NR; user equipment (UE) procedures in idle mode and in RRC inactive state," 3rd Generation Partnership Project (3GPP), Tech. Rep. TS 38.304, Version 15.1.0, Release 15, Oct. 2018.
- [8] "LTE; evolved universal terrestrial radio access (E-UTRA); user equipment (UE) procedures in idle mode," 3rd Generation Partnership Project (3GPP), Tech. Rep. TS 36.304, Version 15.3.0 Release 15, May 2019.
- [9] "Digital cellular telecommunications system (phase 2+) (GSM); universal mobile telecommunications system (UMTS); LTE; vocabulary for 3GPP specifications," 3rd Generation Partnership Project (3GPP), Tech. Rep. TR 21.905, Version 14.1.1, Jul. 2017.
- [10] M. Illian, R. Khalili, A. A. de A. Rocha, and L. Wang, "Cells on autopilot: Adaptive cell (re)selection via reinforcement learning," 2026. [Online]. Available: <https://arxiv.org/abs/2601.04083>
- [11] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. P. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *ACM ICML*, 2016.
- [12] H. Mao, M. Schwarzkopf, S. B. Venkatakrisnan, Z. Meng, and M. Alizadeh, "Learning scheduling algorithms for data processing clusters," in *ACM SIGCOMM*, 2019.
- [13] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Mach. Learn.*, vol. 8, pp. 229–256, 1992.
- [14] H. Mao, S. B. Venkatakrisnan, M. Schwarzkopf, and M. Alizadeh, "Variance reduction for reinforcement learning in input-driven environments," *CoRR*, vol. abs/1807.02264, 2018.
- [15] A. Ramaswamy, "Gradient clipping in deep learning: A dynamical systems perspective," in *ICPRAM*, 2023.
- [16] A. Silva and M. C. Gombolay, "Encoding human domain knowledge to warm start reinforcement learning," in *AAAI*, 2021.
- [17] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *ACM ICML*, vol. 382, 2009.
- [18] M. J. Hausknecht and P. Stone, "Deep recurrent q-learning for partially observable mdps," in *AAAI*, 2015.
- [19] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. A. Riedmiller, "Playing atari with deep reinforcement learning," *CoRR*, vol. abs/1312.5602, 2013.
- [20] K. Jiang, Q. Wang, Y. Xu, and H. Deng, "Observation-time-action deep stacking strategy: Solving partial observability problems with visual input," in *IJCNN*, 2024.
- [21] K. Bechta, C. Grangeat, and J. Du, "Impact of effective antenna pattern on radio frequency exposure evaluation for 5g base station with directional antennas," in *2020 XXXIIIrd General Assembly and Scientific Symposium of the International Union of Radio Science*, 2020.
- [22] H. Miao, J. Zhang, P. Tang, J. Meng, Q. Zhen, X. Liu, E. Liu, P. Liu, L. Tian, and G. Liu, "A survey of new mid-band/fr3 for 6g: Channel measurement, characterization and modeling in outdoor environment," arXiv preprint arXiv:2504.06727 v1, 2025.
- [23] I. R. (VA3IUL), "Understanding noise figure," <https://www.qsl.net/va3iul/Noise/Understanding%20Noise%20Figure.pdf>, 2023, accessed: 2026-01-09.
- [24] E. V. Markova, V. E. Manaeva, E. Zhabankova, D. Moltchanov, P. Balabanov, Y. Koucheryavy, and Y. Gaidamaka, "Performance-utilization trade-offs for state update services in 5g NR systems," *IEEE Access*, vol. 12, pp. 129 789–129 803, 2024.
- [25] Techplayon, "Cell reselection procedure," <https://www.techplayon.com/cell-reselection/>, 2024, accessed: 2026-01-09.
- [26] M. Fadl, "Lte parameter engineering guidelines," <https://www.scribd.com/document/503090909/LTE-Parameter-Engineering-guidelines>, 2025, uploaded to Scribd; Accessed: 2026-01-09.
- [27] P. Nandkumar, "LTE RAN Ericsson-Huawei-Nokia parameter mapping," <https://pdfcoffee.com/lte-ran-ericsson-huawei-nokia-parameter-mapping-byprasadnandkumar-pdf-free.html>, n.d., uploaded via PDFCoffee; Accessed: 2026-01-09.
- [28] NGMN Alliance, "Ngmn recommendation on self-organising networks (son) and operations & maintenance (o&m) requirements," https://www.ngmn.org/wp-content/uploads/NGMN_Recommendation_on_SON_and_O_M_Requirements.pdf, Next Generation Mobile Networks Alliance, Requirement Specification, 2008, accessed: 2026-01-09.
- [29] OpenStreetMap contributors, "OpenStreetMap data retrieved via Overpass API," <https://overpass-api.de>, 2025, data available under the Open Database License (ODbL).
- [30] H. Deng, Q. Li, J. Huang, and C. Peng, "iCellSpeed: increasing cellular data speed with device-assisted cell selection," in *ACM MobiCom*, 2020.
- [31] A. Mahimkar, A. Sivakumar, Z. Ge, S. Pathak, and K. Biswas, "Auric: using data-driven recommendation to automatically generate cellular configuration," in *ACM SIGCOMM*, 2021.
- [32] J. Chuai, Z. Chen, G. Liu, X. Guo, X. Wang, X. Liu, C. Zhu, and F. Shen, "A collaborative learning based approach for parameter configuration of cellular networks," in *IEEE INFOCOM*, 2019.
- [33] A. Mahimkar, Z. Ge, X. Liu, Y. Shaqalle, Y. Xiang, J. Yates, S. Pathak, and R. Reichel, "Aurora: conformity-based configuration recommendation to improve LTE/5G service," in *ACM IMC*, 2022.
- [34] C. Ge, Z. Ge, X. Liu, A. Mahimkar, Y. Shaqalle, Y. Xiang, and S. Pathak, "Chroma: Learning and using network contexts to reinforce performance improving configurations," in *ACM MobiCom*, 2023.
- [35] H. Deng, C. Peng, A. Fida, J. Meng, and Y. C. Hu, "Mobility support in cellular networks: A measurement study on its configurations and implications," in *ACM IMC*, 2018.
- [36] Y. Li, H. Deng, J. Li, C. Peng, and S. Lu, "Instability in distributed mobility management: Revisiting configuration management in 3g/4g mobile networks," in *ACM SIGMETRICS*, 2016.
- [37] Y. Li, Q. Li, Z. Zhang, G. Baig, L. Qiu, and S. Lu, "Beyond 5g: Reliable extreme mobility management," in *ACM SIGCOMM*, 2020.
- [38] M. Kalntis, A. Lutu, J. A. O. Iglesias, F. A. Kuipers, and G. Iosifidis, "Smooth handovers via smoothed online learning," in *INFOCOM*, 2025.
- [39] N. C. Luong, D. T. Hoang, S. Gong, D. Niyato, P. Wang, Y. Liang, and D. I. Kim, "Applications of deep reinforcement learning in communications and networking: A survey," *IEEE Commun. Surv. Tutorials*, vol. 21, no. 4, pp. 3133–3174, 2019.
- [40] H. Mao, R. Netravali, and M. Alizadeh, "Neural adaptive video streaming with pensieve," in *ACM SIGCOMM*, 2017.
- [41] L. Giacomoni and G. Parisi, "Reinforcement learning-based congestion control: A systematic evaluation of fairness, efficiency and responsiveness," in *IEEE INFOCOM*, 2024.
- [42] M. Blöcher, N. Nedderhut, P. Chuprikov, R. Khalili, P. Eugster, and L. Wang, "Train once apply anywhere: Effective scheduling for network function chains running on FUMES," in *IEEE INFOCOM*, 2024.
- [43] H. Zhou, M. H. M. Elsayed, and M. Erol-Kantarci, "RAN resource slicing in 5G using multi-agent correlated Q-Learning," in *PIMRC*, 2021.